



State Pattern



About me

- Shpak Dmytro
- Java Developer
- Andersen Lab
- Fintech



Agenda

What is State Pattern

Solution/Example

Motivation & Problem

Comparison/Takeaways



The Problem

1

Objects whose behaviour changes based on internal state



The Problem

1

Objects whose behaviour changes based on internal state

2

Bloated if/else or switch statements spread across methods



The Problem

1

Objects whose behaviour changes based on internal state

2

Bloated if/else or switch statements spread across methods

3

Adding a new state requires touching many places (fragile code)



The Problem

1

Objects whose behaviour changes based on internal state

2

Bloated if/else or switch statements spread across methods

3

Adding a new state requires touching many places (fragile code)



The Problem

1

Objects whose behaviour changes based on internal state

2

Bloated if/else or switch statements spread across methods

3

Adding a new state requires touching many places (fragile code)

4

Hard to test, reason about, and extend

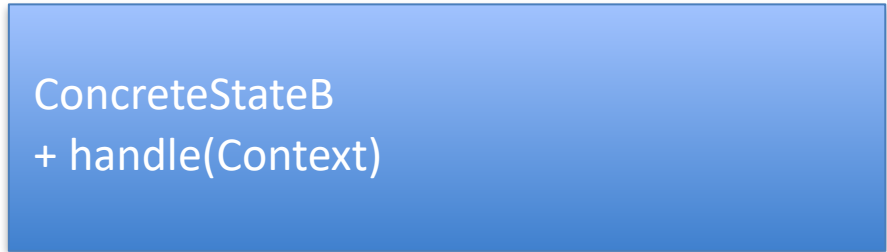
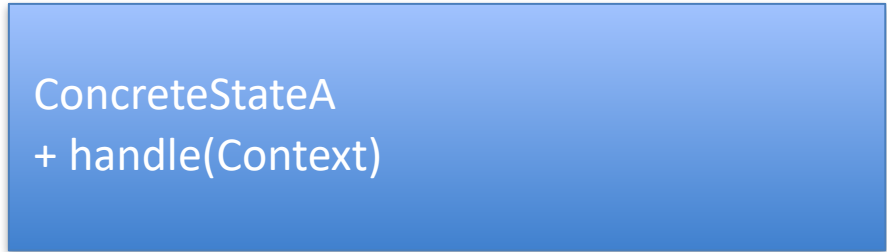
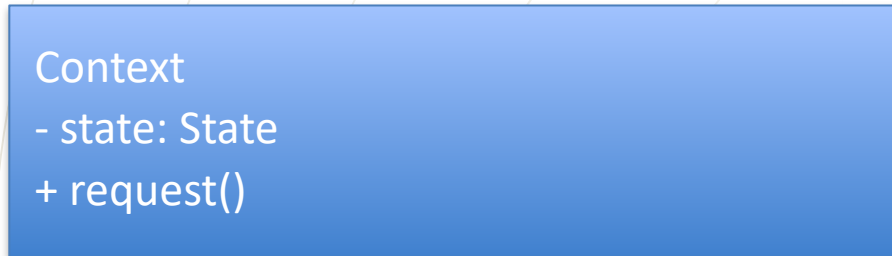


The Solution: State Pattern

- Encapsulate varying behaviour (per state) in separate classes
- Context delegates to a State interface; states are interchangeable
- Open/Closed: Add new states without changing the Context
- Cleaner, testable, and composable design



UML (Simplified)



Smells Indicating State Pattern

- Large conditional logic selecting behavior by status/type
- Flags sprinkled across the class controlling branches
- Frequent changes to add/modify state-dependent behaviour
- Inconsistent transitions (no single source of truth)



Example: Order Lifecycle

- States: NEW → PAID → SHIPPED → DELIVERED → CANCELED
- Operations vary by state: pay(), ship(), deliver(), cancel()
- Invalid operations should be guarded by state



Example

```
public interface OrderState {  
    void pay(Order ctx);  
    void ship(Order ctx);  
    void deliver(Order ctx);  
    void cancel(Order ctx);  
}
```

```
public class Order {  
    private OrderState state;  
    public Order() { this.state = new NewState(); }  
  
    void setState(OrderState state) { this.state = state; }  
  
    public void pay()    { state.pay(this); }  
    public void ship()   { state.ship(this); }  
    public void deliver(){ state.deliver(this); }  
    public void cancel() { state.cancel(this); }  
  
    // domain data, e.g. id, items, totals...  
}
```



Guarding Transitions

1

Each state validates allowed operations

2

Centralises transition logic → fewer bugs

3

Consistent errors/messages per state





Comparison with other patterns

State vs Strategy

- Both use composition + interface
- Strategy: client chooses algorithm; no internal transitions
- State: object changes behaviour over time; states transition
- if transitions matter → State



State vs Observer

- **Observer Purpose** is about communication between objects – specifically a one-to-many relationship where changes in one object (subject) are broadcast to many observer
- Use **Observer** when you need to ensure multiple parts of a system stay in sync with some object's state or events (for example, GUI listeners updating when a data model changes, or an event bus system)
- It's possible to use them together: e.g., an object uses State pattern for its internal modes, and also acts as a Subject in Observer pattern to notify other components whenever its state (mode) changes. But the **goals** remain distinct – Observer is for notification of change, State is for implementing the *logic* of change.
- **State Purpose** is about an object altering its own behavior through internal state objects.
- Use **State** when an object's own behaviour is complex and depends on its mode, and you want to manage that complexity by splitting modes into separate classes.



Real-World Use Cases

- Order/payment lifecycles
- Parser/tokeniser modes
- Workflow engines (ticket → in progress → done)
- Media player (playing/paused/stopped)



Alternatives & Hybrids

- Enums with behaviour (method bodies inside enum constants)
- Table-driven state machines (transitions in data)
- Workflow engines (BPMN) when rules grow complex
- Pattern mix: State + Strategy/Observer/Command





Takeaway

Takeaways

- Replace conditionals with polymorphism
- States own behaviour + transitions
- Simpler tests, safer evolution
- Cleaner, maintainable code
- Great fit for evolving domain workflows



Thank you

- Author: Dmytro Shpak
- My LinkedIn: <https://www.linkedin.com/in/shpak-dmytro/>
- Date: September 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

